

分布式实验报告

题目一：MPI使用梯形法计算连续函数积分

编写基于MPI的并行计算程序，实现对连续函数 $f(x)=\sin(x^2+x)$ ($x \in \mathbb{R}$) 在区间 $[a, b]$ 上的定积分求解。要求使用“梯形法 (Trapezoidal Rule)”进行近似计算

梯形法将积分区间 $[a,b]$ 划分为 n 个等宽子区间，每个子区间宽度为 $h = (b-a) / n$

$$\int_a^b f(x)dx \approx \frac{h}{2} \left[f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right]$$

将总区间 $[a,b]$ 均匀划分为 p 个子区间 (p 为进程数)

每个进程 i 计算其子区间 $[a_i, b_i]$ 的局部积分

通过 `MPI_REDUCE` 将局部积分汇总到主进程

D: > file > study > XDU_CS > 选修 > 分布式计算 > 实验和作业2025 > 4j > MPI > src > TrapezoidalRule.py > ...

```
6     return x * x
7
8 def trapezoidal_rule(a, b, n, func = f):
9     """梯形法计算定积分"""
10    h = (b - a) / n
11    integral = (func(a) + func(b)) / 2.0
12    for i in range(1, n):
13        x = a + i * h
14        integral += func(x)
15    return integral * h
16
17 def main():
18     comm = MPI.COMM_WORLD
19     rank = comm.Get_rank()
20     size = comm.Get_size()
21
22     # 定义积分区间和步数
23     a = 0.0 # 积分下限
24     b = 1.0 # 积分上限
25     n = 1024 # 总的梯形数量
26
27     # 确保梯形数量能被进程数整除
28     if n % size != 0:
29         if rank == 0:
30             print("梯形数量必须能被进程数整除。")
31         MPI.Finalize()
32         return
33
34     # 每个进程计算的梯形数量
35     local_n = n // size
36     h = (b - a) / n # 步长
37
38     # 每个进程的计算区间
39     local_a = a + rank * local_n * h
40     local_b = local_a + local_n * h
41
42     # 每个进程计算其子区间的积分
43     local_integral = trapezoidal_rule(local_a, local_b, local_n, lambda x: np.sin(x*x+x))
44
45     # 所有进程将结果汇总到主进程
46     total_integral = comm.reduce(local_integral, op=MPI.SUM, root=0)
47
48     if rank == 0:
49         print(f"使用 {size} 个进程计算积分结果: {total_integral:.10f}")
50
51     # 在作为模块导入时不运行 main 函数
52     if __name__ == "__main__":
53         main()
```

结果如下:

D:\file\study\XDU_CS\选修\分布式计算\实验和作业2025\4j\MPI\src>mpiexec -n 4 python Test.py

```
Hello from rank 2 of 4
Hello from rank 1 of 4
Hello from rank 0 of 4
Hello from rank 3 of 4
```

D:\file\study\XDU_CS\选修\分布式计算\实验和作业2025\4j\MPI\src>mpiexec -n 4 python TrapezoidalRule.py
使用 4 个进程计算积分结果: 0.6143218007

题目2

输入文件为学生成绩信息，包含了必修课与选修课成绩，格式如下：

班级1, 姓名1, 科目1, 必修, 成绩1

班级2, 姓名2, 科目1, 必修, 成绩2

班级1, 姓名1, 科目2, 选修, 成绩3

.....,,,

编写Hadoop平台上的MapReduce程序，分别实现如下功能：

计算每个学生必修课的平均成绩。

统计每个班级中所有课程（必修+选修）平均成绩排名前五的学生姓名和成绩。

使用两个MapReduce 程序分别实现问题，对于第一个问题，针对输入字符串，Map成<姓名，成绩>只选择必修；接着在Reduce阶段，通过姓名进行分组，以实现问题的并行化处理。

针对第二个问题，则使用两个MapReduce 进行解决，第一个 MapReduce 同问题一，得到每个学生的平均成绩，Map成<（班级名，姓名），成绩>，Reduce输出<（班级名，姓名），平均成绩 >。第二个MapReduce 则根据班级进行Map分类，把每一个班级的学生聚集到同一个类中，这样 就可以在Reduce 时进行排名，通过维护一个大小为5的最小堆进行实现。

```
root@hadoopspark:~/share/4/ScoreAnalyse# cat commands.txt
mvn clean
mvn package

hadoop fs -mkdir input
hadoop fs -put ./grades.txt input
hadoop fs -cat input/grades.txt

// compute the average
hadoop jar ./target/ScoreAnalyse.jar com.org.xidian.MapReduceScoreAverage input/grades.txt output
// sort students by average score
hadoop jar ./target/ScoreAnalyse.jar com.org.xidian.MapReduceSortByClass input/grades.txt temp output

hadoop fs -ls output
hadoop fs -cat output/part-r-00000
hadoop fs -rm -r output
hadoop fs -rm -r temp
```

龚琪山	83.57142857142857
龚瑶娟	79.0
龚盈	68.57142857142857
龚睿	67.14285714285714
龚笑欣	74.0
龚维	79.57142857142857
龚耀	71.0
龚耀显	80.42857142857143
龚良乎	70.28571428571429
龚良迪	76.0
龚艳	70.85714285714286
龚诗思	79.71428571428571
龚诚荣	73.28571428571429
龚豪	68.28571428571429
龚豪坦	75.14285714285714
龚贤	81.14285714285714
龚贤然	88.57142857142857
龚贤诚	77.28571428571429
龚轩耀	78.14285714285714
龚达山	66.85714285714286
龚通	77.14285714285714
龚通灵	74.28571428571429
龚道文	79.14285714285714
龚道正	81.85714285714286
龚锐余	76.0
龚雄民	78.0
龚雨	64.71428571428571
龚雨余	71.71428571428571
龚顺	73.14285714285714

```
曹贤,90.75
蔡丰,88.125
张威彤,87.875
顾思巍,87.11111111111111
顾安,90.5
姜伟帅,89.625
叶丽然,89.33333333333333
唐析维,87.57142857142857
汤欣,86.875
崔姣通,86.77777777777777
邵怡,86.625
江哲,86.57142857142857
邱容,86.42857142857143
沈思,85.9
王顺,89.0
曹俊,87.875
顾荣,87.5
罗山,86.55555555555556
范明欣,86.33333333333333
程义雨,88.57142857142857
贺妯,88.5
邱晓,87.55555555555556
孙盈,86.75
戴华,86.125
魏顺余,89.14285714285714
何锐安,86.77777777777777
周军博,86.0
常哲,85.75
段欣武,85.5
```

题目3

输入文件的每一行为具有父子/父女/母子/母女/关系的一对人名，例如： Tim, Andy

Harry, Alice

Mark, Louis

Andy, Joseph

.....,

假定不会出现重名现象。 编写Hadoop 平台上的MapReduce 程序，找出所有具有grandchild-grandparent 关系的人名对。

通过对关系的重构，<Tim,Andy>的关系变成

<Tim,"C,Andy>与<Andy,"P,Tim>,通过对Key值的合并而成，将所有的父关系用笛卡尔乘积运算，通过父关系来判断祖父与孙子的关系，

```
Crystal Randall
Lillian Emma
Justin Emma
Gillian Emma
Fred Emma
Daisy Bob
Tanya Greta
Angelia Greta
Jill Robinson
Noah Robinson
Jerry Robinson
Heidi Brown
Hank Brown
Katherine Brown
Kimberly Brown
Elvis Carrie
Tina Carrie
Andrew Carrie
Jenna Josephine
Rita Josephine
Nathaniel Josephine
Miranda Josephine
Natalie Leslie
Bruce Leslie
Hunk Leslie
Darcy Geoffrey
Christy Geoffrey
Janet Geoffrey
```

题目4

输入文件为学生成绩信息，包含了必修课与选修课成绩，格式如下：

班级1, 姓名1, 科目1, 必修, 成绩1

班级2, 姓名2, 科目1, 必修, 成绩2

班级1, 姓名1, 科目2, 选修, 成绩3

.....

编写Spark程序，实现如下功能：按班级统计必修课平均成绩在：90-100、80-89、70-79、60-69 和 60 分以下这 5 个分数段的学生人数。

数据重组为 ((班级, 学生), 成绩) 格式，便于后续按学生分组计算

- 按 (班级, 学生) 进行分组
- 对每个学生的所有必修课程成绩求平均值

将数据转换为 ((班级, 分数段), 1) 格式

output分成三个的原因

```
counts = class_bucket.reduceByKey(lambda a, b: a + b)
```

reduceByKey 操作会导致数据重新分区 (shuffle)，最终的分区数决定了输出文件的数量。输出被分成了 3 个文件 (part-00000、part-00001、part-00002)，这表明 counts RDD 最终有 3 个分区

```
-rw-r--r--  1 root supergroup      0 2025-06-12 15:40 output/_SUCCESS
-rw-r--r--  1 root supergroup    383 2025-06-12 15:40 output/part-00000
-rw-r--r--  1 root supergroup    319 2025-06-12 15:40 output/part-00001
-rw-r--r--  1 root supergroup    422 2025-06-12 15:40 output/part-00002
root@hadoopspark:~/share/4/ScoreAnalyse# hadoop fs -cat output/part-r-00000
cat: 'output/part-r-00000': No such file or directory
root@hadoopspark:~/share/4/ScoreAnalyse# hadoop fs -cat output/part-00000
170315班,60-69,162
160315班,60-69,149
170301班,80-89,123
170314班,60-69,137
160311班,80-89,121
170312班,60-69,157
160313班,60-69,168
160314班,80-89,124
170311班,80-89,129
170301班,60-69,140
160312班,90-100,5
170301班,60以下,2
160315班,60以下,2
170314班,90-100,3
160313班,60以下,3
160301班,60以下,1
160312班,60以下,1
160301班,90-100,2
170313班,60以下,1
root@hadoopspark:~/share/4/ScoreAnalyse# hadoop fs -cat output/part-00001
160312班,80-89,134
170313班,70-79,480
170314班,80-89,136
```

```
170314班,70-79,477
170301班,70-79,453
170313班,60-69,149
160315班,80-89,102
170315班,80-89,117
160315班,90-100,1
160311班,60以下,3
170301班,90-100,2
170313班,90-100,1
170315班,60以下,1
root@hadoopspark:~/share/4/ScoreAnalyse# hadoop fs -cat output/part-00002
160315班,70-79,461
160311班,60-69,160
170315班,70-79,490
160313班,80-89,127
170312班,80-89,118
160313班,70-79,493
160301班,70-79,456
160312班,70-79,499
160311班,70-79,442
160314班,70-79,479
160314班,60-69,176
160301班,60-69,146
160301班,80-89,125
170311班,60-69,158
160312班,60-69,163
170314班,60以下,5
160314班,90-100,3
160314班,60以下,4
170311班,60以下,2
```